

Exercise 2: System Design with CASE Tools

Gesture Language Translator

20. CASE Tool Setup

Recommended Tool: StarUML (free) or Visual Paradigm Community Edition.

Download StarUML from <https://staruml.io>, install and create a new project named: Gesture Language Translator.

21. Team Workspace Setup

Since this is a solo project, create a local project folder: GestureTranslator/design/ and save all diagrams inside this folder. Optionally push to GitHub under a /design directory for version tracking.

22. Project Structure in CASE Tool

Create the following diagram folders in StarUML:

GestureTranslator/

 UseCaseDiagrams/

 ClassDiagrams/

 SequenceDiagrams/

 ActivityDiagrams/

 ComponentDiagram/

 DeploymentDiagram/

23. Actors Identification

Actor ID	Actor Name	Actor Type	Description
ACT-01	Deaf/Mute User	Primary	Main user who performs sign language gestures in front of the camera
ACT-02	Hearing User	Primary	Receives and reads the translated text output to communicate back
ACT-03	Camera	System	Hardware component that captures live video frames of gestures
ACT-04	AI/ML Engine	System	Core processing unit that analyzes frames and recognizes gestures
ACT-05	Text-to-Speech Engine	System	Converts translated text into audio output for hearing users

24. Use Case Definition

UC ID	Use Case Name	Actor(s) Involved	Description
UC-01	Open Application	Deaf/Mute User, Hearing User	User launches the web/mobile application
UC-02	Grant Camera Permission	Deaf/Mute User, Camera	User grants the app permission to access the device camera
UC-03	Start Live Video Capture	Camera, AI/ML Engine	System activates the camera and begins capturing live video frames
UC-04	Detect Hand Gesture	Deaf/Mute User, AI/ML Engine	AI engine identifies and classifies hand gestures from video frames
UC-05	Detect Facial Expression	Deaf/Mute User, AI/ML Engine	AI engine identifies facial expressions to support gesture context
UC-06	Translate Gesture to Text	AI/ML Engine	System maps recognized gesture to corresponding text output
UC-07	Display Translated Text	Hearing User	Translated text is rendered on screen in real time
UC-08	Enable Text-to-Speech	Hearing User, TTS Engine	System reads out translated text as audio for the hearing user
UC-09	Stop/Pause Translation	Deaf/Mute User, Hearing User	User pauses or stops the live translation session
UC-10	View Translation History	Deaf/Mute User, Hearing User	User reviews previously translated gestures from the current session

25. Use Case Relationships

UC-03 includes UC-02: Camera cannot start without permission being granted.

UC-04 includes UC-03: Gesture detection requires an active live video feed.

UC-05 includes UC-03: Facial expression detection also requires live video.

UC-06 includes UC-04 and UC-05: Translation uses both gesture and expression results.

UC-07 includes UC-06: Display of text depends on a completed translation.

UC-08 extends UC-07: Text-to-speech is an optional extension of text display.

UC-10 extends UC-07: Translation history is an optional feature of text display.

26. Use Case Documentation

Field	UC-04: Detect Hand Gesture	UC-06: Translate Gesture to Text	UC-07: Display Translated Text
Actor(s)	Deaf/Mute User, AI/ML Engine	AI/ML Engine, Translation Engine	Hearing User, Text Output Module
Preconditions	Camera permission granted; live video feed is active	Hand gesture successfully detected (UC-04 complete)	Translation successfully generated (UC-06 complete)
Main Flow	1. Camera captures frame 2. Frame sent to AI engine 3. AI identifies hand shape and position 4. Gesture label returned	1. Gesture label received 2. Translation engine maps label to text 3. Text string generated 4. Passed to display module	1. Text string received 2. UI renders text on screen 3. Text updated in real time per frame

Alternative Flow	If gesture is unclear, AI returns low confidence and requests user to repeat gesture	If gesture label not found in map, return Gesture not recognized message	If no text available, display Waiting for gesture placeholder
Postconditions	Gesture label is stored and passed to translation engine	Translated text string is ready for display	Text is visible on screen and optionally passed to TTS engine

27. Class Identification

The following key classes have been identified for the class diagram:

User, GestureCapture, FrameProcessor, GestureRecognizer, TranslationEngine, TextOutput, SpeechConverter, TranslationHistory.

28. Class Attributes and Methods

GestureRecognizer

Attributes: modelPath (String), confidenceThreshold (Float), detectedGesture (String)

Methods: loadModel(): void, detectGesture(frame: Frame): String, getConfidence(): Float

TranslationEngine

Attributes: language (String), translationMap (Dictionary)

Methods: translate(gesture: String): String, updateLanguage(lang: String): void

TextOutput

Attributes: displayText (String), timestamp (DateTime)

Methods: displayTranslation(text: String): void, clearDisplay(): void

29. Class Relationships

Rel ID	Class A	Relationship Type	Class B	Description
REL-01	User	Association	GestureCapture	User initiates the gesture capture process
REL-02	GestureCapture	Association	FrameProcessor	Captured video frames are passed to the frame processor
REL-03	FrameProcessor	Aggregation	GestureRecognizer	Frame processor contains and uses the gesture recognizer
REL-04	GestureRecognizer	Dependency	TranslationEngine	Recognized gesture label is sent to the translation engine
REL-05	TranslationEngine	Association	TextOutput	Translated text is passed to the text output module

REL-06	TextOutput	Association	SpeechConverter	Text output optionally triggers speech conversion
REL-07	TextOutput	Aggregation	TranslationHistory	All displayed translations are stored in history
REL-08	User	Association	TextOutput	User views the final translated text on screen

30. Sequence Diagrams

SD-01: Normal Translation Flow

User opens application. App requests camera permission. Permission is granted. User performs gesture. Camera sends frame to FrameProcessor. FrameProcessor sends frame to GestureRecognizer. GestureRecognizer sends gesture label to TranslationEngine. TranslationEngine sends translated text to TextOutput. TextOutput displays text to the user.

SD-02: Unknown Gesture Flow

User performs gesture. Camera sends frame to FrameProcessor. GestureRecognizer returns Unknown label. TranslationEngine passes Gesture not recognized to TextOutput. TextOutput shows error message to user.

SD-03: Text-to-Speech Flow

TranslationEngine sends translated text to TextOutput. TextOutput passes text to SpeechConverter. SpeechConverter plays audio through the device speaker.

31. Object Interactions Documentation

FrameProcessor and GestureRecognizer interact synchronously. Every frame is processed before the next is captured to maintain accuracy.

TranslationEngine and TextOutput interact in real time with minimal delay, targeting under 2 seconds per NFR-02.

SpeechConverter interacts asynchronously. Audio plays after text is displayed on screen.

32. Alternative Flows

Camera denied: App displays message - Please grant camera access to continue.

Low lighting detected: FrameProcessor notifies user - Improve lighting for better detection.

No gesture in frame: GestureRecognizer sends message to TextOutput - No gesture detected in frame.

33. Activity Diagrams

AD-01: Main Translation Workflow

Start. Open App. Grant Camera Permission. If denied, show error and end. If granted, start video feed. Capture frame. Detect gesture. If not detected, loop back to capture frame. If detected, recognize gesture. Translate to text. Display text. If user continues, loop back to capture frame. If not, stop and end.

AD-02: Text-to-Speech Workflow

Start. Receive translated text. Check if TTS is enabled. If not, display text only and end. If yes, convert text to speech. Play audio. End.

34. Workflow Documentation

AD-01 contains a decision point at gesture detection (detected vs not detected) and a continuous loop for real-time capture during a session.

AD-02 contains a decision point at the TTS toggle (enabled vs disabled), allowing the user to choose between silent and audio output modes.

Both workflows support continuous real-time execution throughout a translation session.

35. Component Diagram

Web/Mobile UI communicates with the Video Capture Module. The Video Capture Module feeds into the Frame Preprocessing Module. The Frame Preprocessing Module connects to the Gesture Recognition Module which runs the AI/ML model. The Gesture Recognition Module passes results to the Translation Engine. The Translation Engine connects to the Text Display Module. The Text Display Module optionally connects to the Text-to-Speech Module. Both the Text Display Module and TTS Module connect to the Translation History component.

36. Deployment Diagram

All components are deployed on the User Device (Mobile or Laptop). The device runs the Browser or Mobile App which interfaces with Camera Hardware. The Local AI Model (TensorFlow Lite / MediaPipe) runs entirely on-device. No external server or cloud service is required, ensuring privacy and offline capability.

37. Design Document - Design Decisions

ID	Design Decision	Approach Chosen	Alternative Considered	Rationale
DD-01	AI Processing Location	On-device (local)	Cloud-based processing	Preserves user privacy; no internet dependency; faster response time
DD-02	AI Framework	MediaPipe + TensorFlow Lite	OpenCV only	MediaPipe provides built-in hand landmark detection; TFLite is lightweight for mobile/web
DD-03	Application Platform	Web/Mobile browser-based	Native desktop app	Maximum accessibility; no installation required; works on any device with a camera

DD-04	Input Method	Standard camera	Sensor gloves or depth cameras	No extra hardware needed; aligns with project constraint of no paid tools
DD-05	Architecture Style	Modular layered architecture	Monolithic single-script	Supports maintainability NFR-04; each module can be tested and updated independently
DD-06	Translation Response Time	Real-time per frame under 2 seconds	Batch processing	Meets performance requirement NFR-02; essential for natural communication flow
DD-07	Data Storage	No persistent cloud storage	Database-backed storage	Aligns with out-of-scope decision; protects sensitive user communication data
DD-08	Text-to-Speech	Optional toggle feature	Always-on audio	Gives user control; hearing users may prefer silent text in noisy environments